
Exercise: Hospital Bed, extended edition

In these exercises, you will work with C# threads, the producer – consumer design pattern, GoF Strategy, GoF Observer and unit tests.

Your job is to design and implement software for a hospital bed.

The caretakers at the hospital have trouble with patients leaving their beds, when they are not supposed to.

The hospital has hired *you*, a brilliant young engineer, to solve their problem.

You came up with a solution:

Install a presence sensor and a buzzer in the beds.

If the presence sensor detects, that there is no patient in the bed, alarm the caretakers using the buzzer.

Exercise 1:

Design the software to use for the bed.

Create a UML class diagram and any other diagrams, you find relevant for the design.

Explain your design to one or two of your fellow students.

Exercise 2:

Implement your design as a Console application and create a simulated version of the sensor.

The simulated sensor should be polled, i.e., you have to ask the sensor if a patient is present.

The sensor should provide a random value (true or false).

Simulate the buzzer by printing to the console, when the buzzer is buzzing.

Exercise 3:

Did you use the producer – consumer design?

If not, change your code to use this design. Rewriting your code like this is often called *refactoring*.

Now, you are going to refactor and extend your solution so that it becomes much more flexible. You are also going to implement new ways to alarm caretakers, and finally make it very flexible to select how alarming will take place. This is going to happen by means of the GoF Strategy pattern.

Exercise 4:

Alarming is a candidate for the application of GoF Strategy pattern, so: Update your design (UML) and refactor your application so that alarming uses GoF Strategy. Implement alarming in two ways.

- A buzzer
- A light

How hard would it be to implement a third way of alarming, i.e., a text/email to the caretakers?

Exercise 5:

Redesign your software so that the alarm method i.e., “strategy” can be changed at run-time, so that you can choose the strategy when the program runs. Pressing the ‘B’ key shall make the program use the buzzer and pressing the ‘L’ key shall make the program use the light.

Exercise 6:

A test has shown that the sensor sometimes provides false values, i.e., occasionally it reports false, even though a patient is in bed.

As an engineer, you know that you sometimes have to filter your input signals. So, you come up with a very simple filter: The sensor input must be consistent for 3 consecutive samples before it can be trusted. Update your design to include the filter before the alarm is called. When you do, have the Open Closed Principle in mind. Implement your solution, after you have *updated* your design.

Proposal: Create a separate class for the filter. The method signature could look like

```
bool Filter(bool sample)
```

where the Filter method is called every time, a new sample is read from the sensor. The sample shall be the parameter passed to the method and the method shall return the filtered value. You have to choose, what the initial value should be: absent or present, because you need at least 3 samples to decide the state based on the samples from the presence sensor.

Exercise 7:

Create a Unit test project in your C# solution. Add a unit test for you filter class, and make sure you test cases where the patient is in bed and out of bed, but also when there are changes from one state to another.

Exercise 8:

The filtering is also a candidate for the application of the GoF Strategy. By applying the GoF Strategy, your software can contain two or more types of filtering:

- Pass-through filter, i.e., no filtering
- Noise filter, i.e., 3 consecutive samples before the input is trusted

Update your design and refactor your software for the application of GoF Strategy, i.e., that it contains both ways of filtering and the chosen filter can be changed while the program runs.

How hard would it be to implement and use a third type of filter?

Exercise 9:

Suggest a design that uses GoF Observer, so that the BedControl can notify an observer Alarming, when "Patient presence" has changed. The class Alarming should be responsible for delegating to the Alarm strategy.

Exercise 10:

Implement your design.

Exercise 11:

Extend your design and implementation to introduce a Log class. This class shall log (to screen or file) every time the patient leaves his/her bed. Discuss how this new functionality can be introduced into your design. What classes need to change (if any)? Is it easy or hard to introduce this change?

Exercise 12 (advanced, optional)

One disadvantage in the current design is that it does not lend itself to multithreading very well, i.e. if BedController and Alarming should run in separate threads. Think about it: If "Patient presence changed" occurs, the BedController calls Notify() on Alarming, which then retrieves the state of the patient presence and sounds/silences the alarm. All this, however, occurs in the BedController's thread, since this is the thread that calls Notify() in the first place. The intention was that the handling of the event should occur in Alarming's thread.

Your mission, *should you choose to accept it*, is to change the implementation so that the *raising* of the event occurs in the BedController's thread, but the actual *handling* takes place in Alarming's thread. Hint: The registration of the event occurrence and the handling of it is not necessarily the same thing.